

Programmation

Le programme se compose de trois fonctions : la fonction `computeSineSum`, la fonction `runType`, et la fonction `main`.

La fonction `computeSineSum` est assez simple, elle prend en entrée une table de nombre et retourne la somme de leur sinus. Elle initialise la somme à zéro, passe à travers chaque entrée de la table en argument, et pour chaque entrée elle calcule le sinus de l'entrée et l'ajoute à la somme, et finalement elle retourne la somme finale.

La fonction `main` sert uniquement à lire le flag avec lequel le programme a été appelé et le passé en argument à `runType`. Le contenu de `runType` a été séparé de la fonction `main` afin de pouvoir plus facilement faire des tests sur les différents types de flag. `main` permet aussi de lancer une erreur si le type passé en flag est retourné comme invalide par `runType`.

Finalement, la fonction `runType` contient le cœur du programme. Elle est séparée en deux parties selon le type de données passé en flag, puisque cette distinction change les fonctions qu'on utilise pour manipuler les données, mais la structure des deux parties est la même. La fonction commence par créer une tranche de tableau de la taille voulu (cette valeur est gardée dans une constante au début du fichier pour qu'il soit facile de rapidement la modifier). Ensuite la fonction remplit cette tranche avec des valeurs aléatoires dans l'intervalle demandées dans l'énoncé pour chaque type. Une fois les données prêtes, la fonction note le temps avant l'usage de `computeSineSum`, puis l'appelle sur les données générées, et, finalement, elle note le délai depuis le temps noté. Ce délai est le temps que le programme a passé dans la fonction `computeSineSum` pour les données qui lui ont été passées en argument. Si `runType` reçoit en argument `int` ou `float`, elle exécute la structure décrite et ne retourne aucune erreur, mais si un autre type est passé en argument, elle retournera une erreur explicitant que ce type est invalide.

En ce qui concerne la programmation des tests, on a commencé par les tests sur les fonctions `computeSineSum` et `runType` avant de programmer les benchmarks. Tous les scénarios qui sont censés fonctionner ont été testés pour les deux types de données possible, soit `int` et `float64`. On a donc testé la fonction avec de petites quantités des données, avec des tableaux de trois entrées; pour de grandes quantités de données, avec des tableaux de 100 000 entrées; et pour aucune donnée, avec des tableaux vides. Dans tous les cas sauf ce dernier, on n'a pas directement comparé le résultat attendu avec le résultat obtenu par `computeSineSum`, mais on a plutôt vérifié que leur différence était plus petite de 10^{-9} , c'est-à-dire qu'elle était sans conséquence dans le contexte. Cette marge est permise à cause des variations qui se produisent lors de la manipulation de nombres à virgule encodés sous forme binaire. Tant que ces variations sont sans conséquences pour les utilisateurs du programme, elles ne constituent pas une erreur.

Pour les tests sur `runType`, les scénarios inclus un type qui n'a aucun rapport avec le programme, les deux types attendus `float` et `int`, et finalement un type attendu, mais en majuscule. Ce dernier scénario doit retourner une erreur puisque les seules entrées qui doivent être acceptées sont exactement `int` et `float`. On ne teste pas pour l'absence de flag avec cette fonction, puisque le flag est programmé pour avoir une valeur par défaut de `float`, donc si aucune valeur n'est passée en

ligne de commande, la fonction runType recevra l'argument float et non un argument vide.

Pour les benchmarks, on les a séparés par type de données pour la même raison que runType est séparé. De plus, on utilise une structure indépendante des fonctions de benchmark afin de stocker les noms des tests et la valeur de pourcentage de donnée qu'on test pour ce test. Cette structure nous permettra de plus facilement itérer sur les différents pourcentages demandés dans l'énoncé et de référer à chaque test dans le résultat. Chaque benchmark commence par initialiser et peupler une tranche de tableau de la taille passée en constante dans le programme principal. Ensuite, on passe à travers chaque valeur de pourcentage à tester. Pour chacune d'entre elles, on utilise la commande Run afin de répéter la fonction computeSineSum un nombre de fois déterminer dynamiquement sur une tranche du tableau de donnée contenant uniquement les premières n% entrées, où n est le pourcentage testé courant.

Résultats

Table 1. Benchmark sur le type int

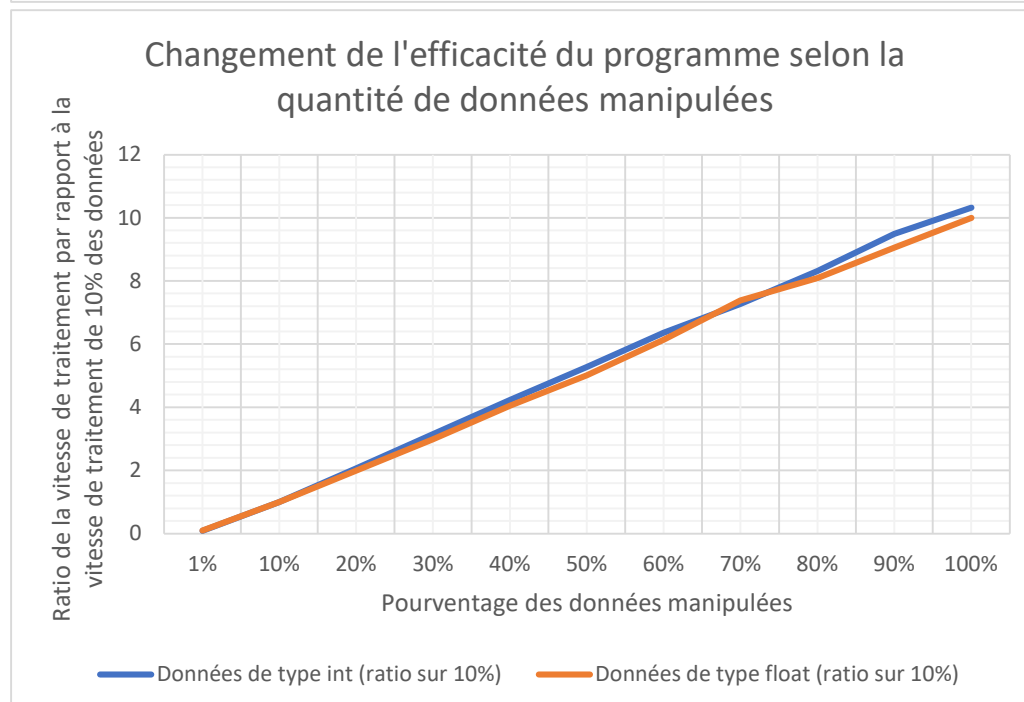
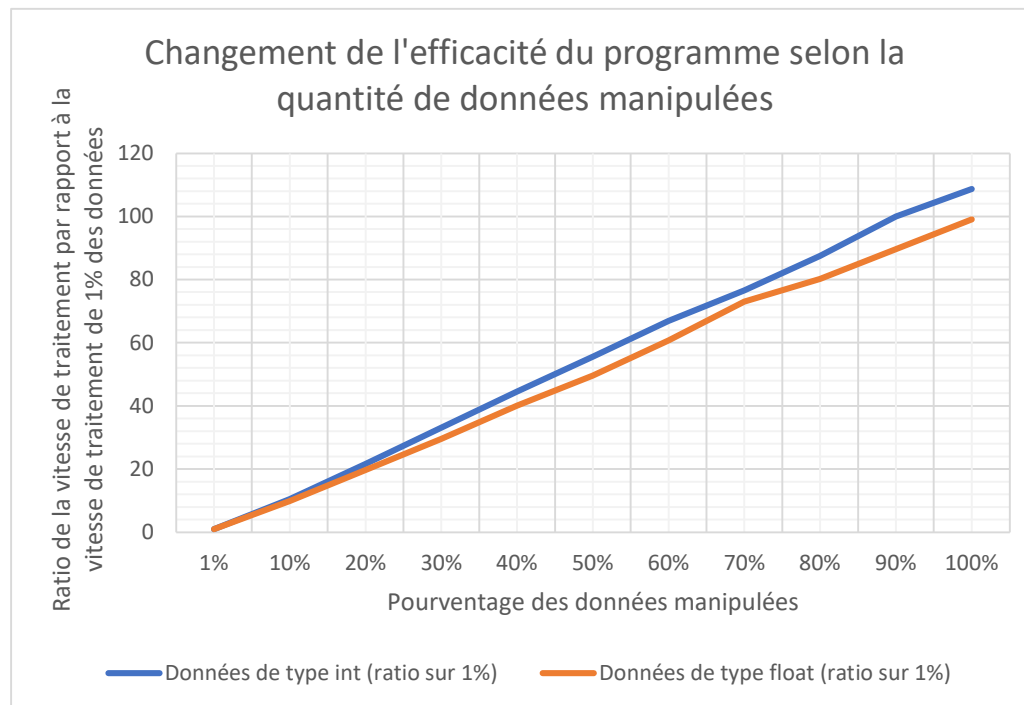
Pourcentage des données utilisées	Nombre d'itérations	Temps par opération	Octets par opération	Allocations par itération
1%	7113	149559	0	0
10%	753	1575127	0	0
20%	373	3242201	0	0
30%	247	4952978	0	0
40%	182	6667999	0	0
50%	147	8311401	0	0
60%	120	10013707	0	0
70%	100	11445593	0	0
80%	93	13091229	0	0
90%	84	14949007	0	0
100%	69	16257706	0	0

Table 2. Benchmark sur le type float64

Pourcentage des données utilisées	Nombre d'itérations	Temps par opération (ns/op)	Octets par opération (o/op)	Allocations par itération (allocs/op)
1%	14893	79090	0	0
10%	1556	783309	0	0
20%	770	1558492	0	0
30%	507	2340584	0	0
40%	370	3170870	0	0
50%	309	3921387	0	0
60%	254	4804279	0	0
70%	207	5777832	0	0
80%	177	6339170	0	0
90%	168	7089050	0	0
100%	148	7834451	0	0

Si on observe les résultats bruts, ils semblent être logiques. En effet, il est normal que plus on traite de donnée, plus le traitement est long. Similairement, si on observe les ratios entre chaque tranche, il est logique d'observer une multiplication par environ 10 entre les temps pour 1% et 10%, et par environ 2 entre les temps pour 10% et 20%, et ce, dans chacune des tables.

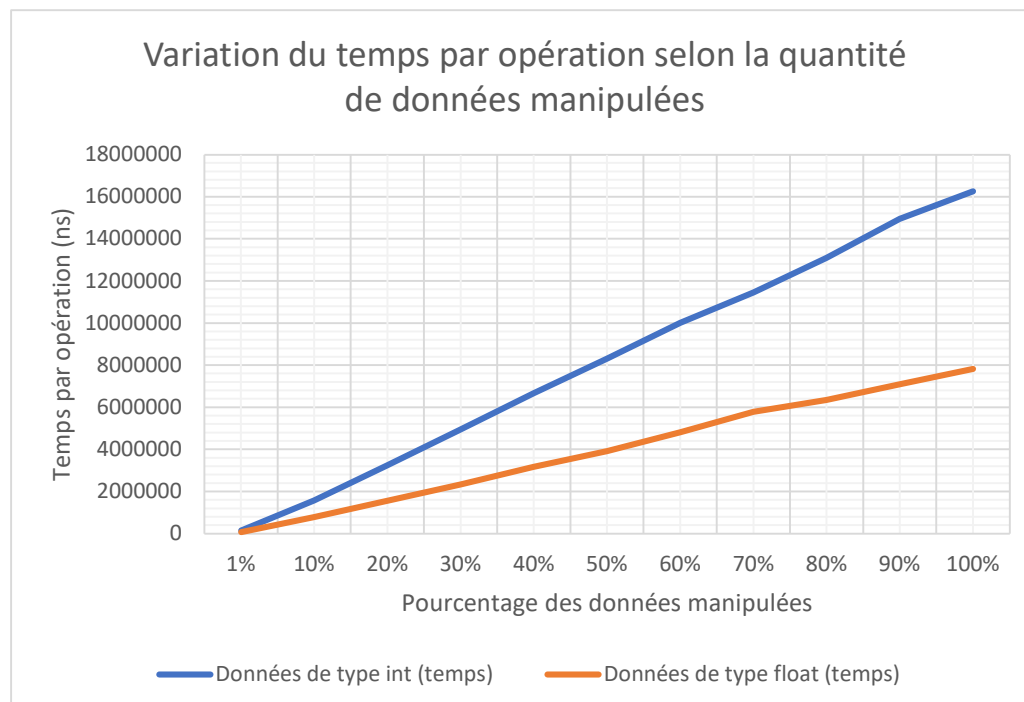
Cependant, si on observe plus granulairement les ratios à l'intérieur de chaque tableau individuellement, on remarque que, pour les données int, les ratios entre



les temps dépassent le ratio entre les données, et cet écart augmente plus la quantité de données traitées augmente. Par exemple, le ratio de temps entre 1% des données et 10% des données est ≈ 10.53 , alors que le ratio entre 1% et 10% est 10, mais le ratio de temps entre 1% et 90% est ≈ 99.95 alors que le ratio entre 1% et 90% est 90. Cet écart de ratio est aussi remarquable si on base le ratio sur la valeur de temps à 10%. Cependant, les données de format float ne souffrent pas autant de ce problème, et le ratio de temps est même parfois plus petit que le ratio de donnée. Par exemple, le ratio de temps entre 1% et 90% des données en float est ≈ 89.63 , alors qu'on pourrait s'attendre à un ratio de 90 dans le meilleur des cas.

Si on se penche sur les temps en tant que tels, on remarque que les données de type float sont beaucoup plus efficaces que les données de type int. Si on compare les temps par opération bruts, on voit rapidement que les données de type float sont manipulées beaucoup plus rapidement que les données de type int pour une même quantité de données. Les données int prennent environ deux fois plus de temps que les données de types float à être manipulées. Si on prend les temps obtenus avec les benchmarks et qu'on les divise par les pourcentages de 1 000 000 auxquels ils correspondent, on obtient les temps moyens par sinus pour chaque tranche. Si ensuite on fait la moyenne de ces moyennes, on peut estimer le temps que le calcul d'un sinus prend avec chaque type de données. On se retrouve donc avec un temps moyen par sinus de $\approx 16.272\text{ns}$ pour les données en int, et de $\approx 7.909\text{ns}$ pour les données en float. De plus, ces moyennes de temps sont plus variables pour les données en int qu'en float, avec une variation de $\approx \pm 0.867\text{ns}$ des moyennes pour les données en int et $\approx \pm 0.231\text{ns}$ des moyennes pour les données en float.

Ces variations pourraient être expliquées par le fait que les données en int entrées dans la fonction `math.Sin` doivent être converties en float avant d'être utilisées par la fonction, alors que les données float sont déjà du bon type. De plus,



les valeurs des tables de int contiennent des valeurs beaucoup plus grandes que les tables de float, qui ont des valeurs dans $[0,1)$. Puisque ces valeurs sont si grandes, la fonction `math.Sin` doit les réduire par arithmétique modulaire pour les faire entrer dans l'écart sur lequel la fonction donne un résultat, soit $[0, 2\pi]$. Les fonctions trigonométriques sont cycliques, c'est-à-dire que les résultats sur un écart défini se répètent à l'infini dans les deux sens. Par cette propriété, il est plus facile de trouver la valeur équivalente dans le plus petit écart positif possible, $[0, 2\pi]$, afin de calculer le sin que de manipuler une grande valeur. Cependant, la transformation d'un grand nombre en sa congruence mod 2π , est une opération supplémentaire que la vaste majorité des valeurs aléatoires int dans $[0, 1000]$ doivent effectuer, alors que les valeurs float dans $[0,1)$ n'ont pas à être calculés mod 2π puisqu'elles sont déjà dans l'intervalle $[0, 2\pi]$.

Considérant les variations en temps selon le type de données, on utilise les moyennes extraites afin de faire les calculs de distance de la lumière et de sinus par tick. Pour la distance de la lumière, si les données sont en int, on obtient que la lumière parcourt $299792458 \text{ m s}^{-1} \div 10^9 \text{ s ns}^{-1} \times 16.272 \text{ ns} \approx 4.878 \text{ m}$ durant le calcul d'un seul sinus. Si les données sont en float, la lumière parcourt $299792458 \text{ m s}^{-1} \div 10^9 \text{ s ns}^{-1} \times 7.909 \text{ ns} \approx 2.371 \text{ m}$ durant le calcul d'un sinus. De plus, si un tick a une durée de $1/120 \text{ s}$, soit $10^9/120 \text{ ns}$, on peut calculer $\left\lfloor \frac{10^9}{120} \div 16.272 \right\rfloor = 512127$ sinus si les données sont de format int, et $\left\lfloor \frac{10^9}{120} \div 7.909 \right\rfloor = 1053651$ sinus s'elles sont de format float.

Annexe

Annexe 1 - Benchmark Ints Console output

Running tool: C:\Program Files\Go\bin\go.exe test -test.fullpath=true -benchmem -run=^\$ -bench ^BenchmarkSineSumInts\$ TN4

```
goos: windows
goarch: amd64
pkg: TN4
cpu: 11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz
BenchmarkSineSumInts/1%-16      7113      149559 ns/op      0 B/op      0
allocs/op
BenchmarkSineSumInts/10%-16      753      1575127 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/20%-16      373      3242201 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/30%-16      247      4952978 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/40%-16      182      6667999 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/50%-16      147      8311401 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/60%-16      120      10013707 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/70%-16      100      11445593 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/80%-16      93      13091229 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/90%-16      84      14949007 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumInts/100%-16      69      16257706 ns/op      0
B/op      0 allocs/op
PASS
ok      TN4 16.993s
```

Annexe 2 - Benchmark Floats Console output

Running tool: C:\Program Files\Go\bin\go.exe test -test.fullpath=true -benchmem -run=^\$ -bench ^BenchmarkSineSumFloats\$ TN4

```
goos: windows
goarch: amd64
pkg: TN4
cpu: 11th Gen Intel(R) Core(TM) i7-11700K @ 3.60GHz
BenchmarkSineSumFloats/1%-16      14893      79090 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumFloats/10%-16      1556      783309 ns/op      0
B/op      0 allocs/op
BenchmarkSineSumFloats/20%-16      770      1558492 ns/op      0
B/op      0 allocs/op
```

BenchmarkSineSumFloats/30%-16	507	2340584 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/40%-16	370	3170870 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/50%-16	309	3921387 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/60%-16	254	4804279 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/70%-16	207	5777832 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/80%-16	177	6339170 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/90%-16	168	7089050 ns/op	0
B/op 0 allocs/op			
BenchmarkSineSumFloats/100%-16	148	7834451 ns/op	0
B/op 0 allocs/op			
PASS			
ok TN4 18.714s			